

Exploring the ESP32 as an Attack Platform

Afonso Cruz
Networks and Systems Security
up202204984@up.pt

Guilherme Rodrigues
Networks and Systems Security
up202208878@up.pt

Tomás Kulzer
Networks and Systems Security
up202207332@up.pt

Abstract—This project evaluates the ESP32 microcontroller’s capacity to function as a platform for network intrusions. Although the ESP32 is constrained by limited memory, processing power, and a single 2.4 GHz radio, it allows for several offensive security techniques. To define the practical offensive capabilities of this low-cost device, the study details the implementation of several attack modules, which include deauthentication, WPA/WPA2 handshake capture, DHCP starvation, Rogue Access Point (Evil Twin) deployment and ARP spoofing. Although modern security advancements like WPA3 and Protected Management Frames (PMF) neutralize certain technical vulnerabilities, the slow transition to WPA3 and the continued effectiveness of psychological exploits (phishing) mean that the ESP32 remains an interesting tool in modern offensive security. The work demonstrates both the practical offensive potential and the hardware limitations of commodity microcontrollers in modern wireless security research.

I. INTRODUCTION

The ESP32, present in over a billion IoT devices worldwide, is a cheap microcontroller with integrated 2.4 GHz Wi-Fi, which represents an intriguing attack vector.

Despite its constraints, including limited SRAM, a single radio, and the lack of 5 GHz support, the ESP32 provides features such as promiscuous mode and raw 802.11 frame transmission [1]. This research explores the potential of the ESP32 as a platform for implementing wireless attack techniques.

The study focuses on some attack modules including deauthentication, handshake capture, DHCP starvation, Rogue Access Point (AP), and ARP spoofing. The objective is to understand how these attacks operate and what practical limitations arise when they are implemented on constrained hardware. By analyzing successful attacks and implementation limitations, this work highlights the importance of defenses such as Protected Management Frames and user awareness.

A. Context and Problem

While previous open-source projects have demonstrated the viability of using the ESP32 for network attacks, such as deploying Rogue Access Points for credential harvesting, the device is bound by strict hardware. Relying on a single ESP32 presents significant constraints when attempting complex intrusions. For example, the device is restricted by: A low-power CPU that renders on-device cryptographic cracking infeasible; A 520 KB SRAM limit that leads to rapid buffer exhaustion during routing attacks, such as ARP spoofing; A single 2.4 GHz radio module that can only transmit on one channel at a time. As a result, attacks that are theoretically simple

may become unreliable, unstable, or only partially effective when implemented on this platform. This raises an important question: which attacks can realistically be performed using an ESP32, what limitations appear in practice, and which defenses are effective against them?

B. Goals

The goal of this project is to experimentally evaluate the ESP32 as a low-cost platform for wireless and local-network security research. Several attack modules were implemented and tested, including deauthentication, handshake capture, DHCP starvation, rogue access point deployment, and ARP spoofing. For each module, the objective was to understand how the attack works, reproduce it on the ESP32, analyze the results, identify the hardware or software limitations encountered, and discuss possible mitigations.

II. NETWORK ATTACKS

Several open-source projects [2], [3] have demonstrated the viability of deploying Rogue Access Points on the ESP32 for credential harvesting. Of particular relevance to our research is the implementation detailed in [3], which leverages the microcontroller’s built-in dual-mode Wi-Fi radio (WIFI_AP_STA). This architectural approach allows the device to operate concurrently as both a station (client) and an access point which allows for continuous background network scanning while simultaneously hosting a configuration or malicious AP.

The ESP32-ARP-Spoof repository by Tanjib provides a MicroPython implementation of ARP spoofing that can intercept unencrypted traffic within an SSID and cause connectivity issues for victim devices [4].

Prior work has also explored Wi-Fi deauthentication attacks on the ESP32 platform. For example, the ESP32 Wi-Fi Penetration Tool by Stehlík demonstrates that the ESP32 can be used to implement several 802.11 attacks, including deauthentication, by exploiting its ability to transmit crafted Wi-Fi frames through the ESP-IDF wireless stack [5]. This project shows that deauthentication attacks are a common proof-of-concept for ESP32-based Wi-Fi security tools, mainly because they require only management-frame injection and channel synchronization, rather than full packet forwarding or higher-layer protocol manipulation.

Furthermore, our research explores network infrastructure denial-of-service and cryptographic reconnaissance by developing custom modules inspired by established open-source methodologies. Specifically, the engineering of our DHCP

Starvation module, designed to exhaust access point IP pools, was guided by the conceptual approach of the *DHCPGlutton* project [6]. Similarly, our methodology for the passive interception of WPA/WPA2 4-way handshakes drew architectural insights from Electro-Gamma’s framework [7], which we adapted to efficiently filter and isolate EAPOL packets for offline password derivation.

Deauthentication (Deauth)

A deauthentication attack targets the management frames used by Wi-Fi networks. In normal operation, these frames are sent by an access point or client to terminate a connection. However, in WPA/WPA2 networks where Protected Management Frames (PMF / IEEE 802.11w) are not enabled, these frames are not encrypted. As a result, an attacker can forge deauthentication frames and send them to a victim machine.

When a client receives these forged frames, it interprets them as valid requests to disconnect from the network. This can be used to deny service to a specific client, trigger a new WPA/WPA2 four-way handshake that may be captured, or make the victim connect to an Evil Twin.

The attack was implemented on the ESP32 by transmitting raw IEEE 802.11 deauthentication frames. Each frame contained the target MAC address as the receiver address, while the transmitter BSSID field was set to the BSSID of the target AP. This causes the victim device to interpret the frame as if it had been sent by the legitimate router.

The ESP32 transmitted these frames using the ESP-IDF function `esp_wifi_80211_tx()`, which allows raw 802.11 frame injection.

An automatic mode was implemented, in which nearby access points were scanned, a target network was selected, connected clients were detected using promiscuous mode, and the deauthentication attack was then launched against the selected victim.

Tests were performed in a controlled environment using authorized devices and a 2.4 GHz Wi-Fi network. In WPA/WPA2 networks where Protected Management Frames were not enabled, the victim machine disconnected after receiving repeated forged deauthentication frames. The time required for disconnection varied depending on the distance between the ESP32, the victim device, and the access point. In contrast, when the network used WPA3, the victim machine did not disconnect, since management frames are protected and forged deauthentication frames were not accepted.

The main mitigation against deauthentication attacks is enabling Protected Management Frames, defined in IEEE 802.11w [8]. PMF protects management frames such as deauthentication and disassociation frames, preventing attackers from successfully forging them. WPA3 requires PMF, while WPA2 networks may support it as an optional configuration.

Handshake Capture

The security of WPA/WPA2-PSK relies on a 4-way handshake. To execute an offline dictionary attack, our implemen-

tation passively captures Message 1 (AP’s ANonce) and Message 2 (client’s SNonce and MIC). Operating in promiscuous mode on the 2.4 GHz band, the firmware filters for EAPOL packets (EtherType `0x888E`) and parses their Key Information bit flags to isolate these frames. To guarantee the generation of these handshakes on demand, a concurrent Deauthentication attack can be performed.

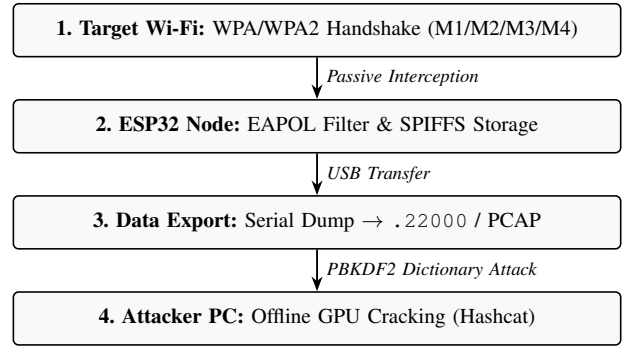


Fig. 1. Data flow pipeline: From passive EAPOL interception on the ESP32 to offline cryptographic cracking on an external host machine.

As illustrated in the data flow pipeline in Fig. 1, the ESP32 acts strictly as a reconnaissance node, writing the captured data to its SPIFFS storage. The microcontroller’s computational limits prohibit on-device cracking, requiring data export via serial dump and conversion for computationally-capable offline machines. Finally, the lack of true “monitor mode” forces the hardware to drop frames with invalid Frame Check Sequences (FCS), potentially leading to missed captures in busy RF environments.

DHCP Starvation

DHCP Starvation is a Layer 2 (targets the Data Link layer) denial-of-service (DoS) attack designed to exhaust the pool of assignable IP addresses on a network’s DHCP server. By completely depleting the available address space, the attacker prevents new, legitimate clients from obtaining network connectivity. The core mechanism of this attack relies on MAC address spoofing: the malicious device changes its physical address and requests a new IP lease for each generated MAC, tricking the server into believing a flood of unique clients is connecting to the network.

The primary challenge of implementing this attack on the ESP32 was dynamically altering the ESP32’s MAC address at runtime. The ESP32’s driver prohibits MAC changes while the Wi-Fi interface is active or connected. Therefore, the starvation cycle required a specific sequence: disconnecting from the current session, modifying the MAC address via `esp_wifi_set_mac()`, and immediately re-initiating the connection. To bypass rudimentary MAC filtering, the spoofed addresses were generated with the “locally administered” bit set (the second-least significant bit of the first byte), marking them as software-generated rather than vendor-assigned OUI addresses.

During testing, we encountered significant timing and stability issues regarding the DHCP routine. If the ESP32 attempted to cycle connections too rapidly, the internal state machine became inconsistent, leading to connection timeouts and memory leaks. To ensure stability, a full Wi-Fi stack restart was forced every 100 reconnections to mitigate heap fragmentation.

To evaluate the attack's efficiency, we tested different cooldown intervals between reconnection attempts. An aggressive 500ms delay exhausted a standard pool of 253 IP addresses in approximately 2 minutes, but occasionally overwhelmed the Access Point (AP), triggering internal rate-limiting defenses. A 1000ms delay proved to be the optimal balance, steadily consuming one IP per second without triggering AP throttling, resulting in complete pool exhaustion in under 5 minutes. Once the attack succeeded, legitimate devices attempting to connect failed to obtain an IP configuration, effectively neutralizing network access. Furthermore, because DHCP servers allocate IPs with a predefined lease time, typically lasting several hours, the attacker can completely stop the ESP32 once the pool is exhausted. The network will remain incapacitated long after the active attack has ceased, as the router will not free the hijacked IP addresses until their respective rogue leases naturally expire.

Rogue AP Module (Evil Twin)

In our implementation of this module, we adopted an architecture similar to [3], leveraging the ESP32's native dual-mode Wi-Fi radio capability (`WIFI_AP_STA`). This allows the microcontroller to simultaneously operate as a station (client) for active reconnaissance and as an access point to host the rogue network.

Upon initialization, the ESP32 broadcasts a hidden management AP. The attacker authenticates to this network to access an administration control panel while the device performs active 802.11 probe scans across all 2.4 GHz channels, logging nearby SSIDs, MAC addresses (BSSIDs), channels, and signal strengths (RSSI). From the control panel, the attacker selects a legitimate AP to clone and configures the target authentication parameters.

Once a target is selected, the ESP32 transitions to a Software AP (SoftAP) mode. Crucially, it mirrors not only the target SSID and authentication type (Open or WPA2-PSK) but also the exact RF channel. By mirroring the channel and positioning the device to provide a higher RSSI than the legitimate AP, the attack exploits the automated roaming and auto-connect behaviors of modern client devices. A local DHCP server is established, assigning connecting victims IP addresses within an isolated private subnet (e.g., `192.168.4.0/24`) and designating the ESP32 as the gateway.

To intercept victim traffic, the firmware implements a captive DNS server on port 53. A wildcard rule forces all DNS A-record queries to resolve directly to the ESP32's IP address. To ensure the phishing payload is presented immediately, the rogue AP intercepts known OS-level captive portal detection endpoints (such as Android's `/generate_204` or Apple's

`/hotspot-detect.html`). By serving the phishing page instead of the expected HTTP 204 or XML responses, the victim's operating system is forced to natively open a captive portal browser window.

The HTTP server on port 80 dynamically serves phishing templates based on the cloned network's configuration. The server utilizes an `%SSID%` placeholder in the HTML, which is replaced at runtime with the actual target network's name to increase legitimacy. We implemented two distinct attack vectors:

- **Password-Protected Networks:** We designed an ISP verification template mimicking a legacy Portuguese ISP ("ZON"). It prompts the user to input their email and password to "restore internet connectivity." This narrative can be easily adjusted to match the target's actual ISP.
- **Open Networks:** The payload presents a "Free Wi-Fi" registration portal requiring the user's full name, email, phone number, and a password. This approach capitalizes on user password reuse across different online services.

Notably, the firmware supports the upload of custom HTML templates at runtime. Attackers can upload tailored HTML files directly to the ESP32's SPI Flash File System (SPIFFS) via the admin panel, allowing for rapid pivoting without the need to recompile or flash the firmware.

When a victim submits their information, the HTTP POST request is parsed, and the credentials, along with the target SSID and victim IP, are persistently logged to a `/creds.txt` file on the SPIFFS storage, ensuring they survive power cycles. The victim is shown a brief success message. However, a key technical constraint of this implementation is the lack of internet passthrough; victims receive no actual connectivity. Furthermore, without being chained to an active deauthentication attack, this module relies entirely on signal strength competition to attract clients.

ARP Spoofing

ARP spoofing is a Man-in-the-Middle (MitM) technique that intercepts traffic between a target device and the network gateway by poisoning their Address Resolution Protocol (ARP) caches. In this project, the ESP32 operates as a malicious actor that maps its own MAC address to the gateway's IP address, forcing the target's outbound traffic to flow through the microcontroller.

Implementing this on the ESP32 presented significant challenges, primarily due to hidden restrictions in the closed-source ESP's Wi-Fi driver. The driver contains a security filter, `ieee80211_raw_frame_sanity_check()`, which silently discards raw 802.11 frames when the device is operating in Station (STA) mode, even while deceptively returning a success code upon transmission. To successfully inject frames, we had to implement a software bypass by overriding this function to always return 0, which required specific linker flags to resolve the duplicate symbol.

Once raw injection was unlocked, strict frame formatting was required to bypass the Access Point (AP) security fea-

tures, such as Client Isolation and IP Spoofing Detection. Our successful attack payload utilized ARP Replies formatted as Gratuitous ARP (where the Sender IP equals the Target IP) to universally force operating systems to update their caches. Furthermore, these frames were transmitted using a broadcast destination MAC address, which forces the AP to flood the malicious frames to all connected stations, effectively bypassing client-to-client unicast isolation.

To evaluate the impact of the attack, we monitored the ESP32's packet forwarding reliability, which exposed severe hardware limitations. Primarily due to the device's limited SRAM and the strict timing constraints of executing the promiscuous callback within a Wi-Fi interrupt context, the microcontroller proved entirely incapable of effectively processing and forwarding the intercepted traffic. Regardless of the gratuitous ARP transmission frequency, packets were consistently dropped during ICMP (ping) tests. Successful ping responses only occurred when legitimate ARP requests from other network devices temporarily restored the correct MAC mappings on the victim's machine, effectively breaking the MitM state.

Given the constant stream of background traffic typically generated by a modern PC, such as routine DNS queries and system telemetry, the ESP32's maximum forwarding capacity is instantaneously exceeded. As a result of this inevitable buffer exhaustion and subsequent Watchdog Timer crashes, this ESP32-based ARP spoofing implementation fails to act as a stable, stealthy Man-in-the-Middle interceptor. Instead, it practically functions as a localized Denial of Service (DoS) attack, completely crippling the victim's network connectivity.

III. ESP32 LIMITATIONS

As demonstrated in the previous modules, pushing the ESP32 to its limits revealed strict hardware and software boundaries:

- **Radio & RF Constraints:** The device is restricted to the 2.4 GHz band with a low transmission power, limiting its effective attack range. Furthermore, the lack of a true "monitor mode" prevents capturing frames with invalid FCS.
- **Processing & Memory:** The 520 KB SRAM capacity causes rapid buffer exhaustion during active routing (e.g., ARP Spoofing). Additionally, its low-power CPU mandates offline execution for computationally heavy tasks like PBKDF2 derivation.
- **Driver & Stability:** Espressif's closed-source driver requires complex linker-level bypasses for raw frame injection and forced interface restarts for MAC spoofing. Prolonged execution of these modules inevitably causes heap fragmentation, requiring programmed reboots.

IV. OVERVIEW WITH NOWADAYS NETWORKS

The attack modules in this research primarily targeted WPA2-PSK, which is the most deployed wireless standard. However, modern architectures pose significant barriers to

these techniques. For instance, WPA3 does not use the vulnerable 4-way handshake, replacing it with the SAE protocol and enforces Protected Management Frames (PMF), effectively neutralizing cracking and deauthentication attacks. Similarly, enterprise environments utilizing RADIUS and DHCP Snooping would rapidly isolate the MAC-spoofing required for DHCP Starvation.

Despite these advancements, ESP32-based attacks remain a contemporary threat. The global transition to WPA3 is slow, leaving millions of networks vulnerable. Furthermore, modules like the Rogue Access Point bypass encryption entirely by exploiting human psychology rather than mathematical flaws. Given its portability and low cost, the ESP32 continues to be an interesting attack vector proving that low-cost microcontrollers can be formidable tools in modern offensive security.

V. CONCLUSION

This project evaluated the ESP32 as a low-cost platform for experimenting with wireless attacks. The implemented modules show that the device can reproduce several attacks, including deauthentication, handshake capture, DHCP starvation, Rogue AP deployment, and ARP spoofing. However, the experiments also show that the ESP32 is limited by its hardware and driver constraints, such as the single 2.4 GHz radio, limited SRAM, lack of true monitor mode, and instability under high packet injection or forwarding loads.

The results indicate that attacks such as deauthentication remain effective against WPA/WPA2 networks without Protected Management Frames, but fail when PMF is used. Other modules, such as handshake capture and ARP spoofing, were possible but constrained by the ESP32's processing, memory, and packet-handling limitations. Overall, the ESP32 proved useful as an educational platform for understanding how these attacks work in practice, while also demonstrating why modern mitigations and hardware limitations significantly affect their reliability.

VI. REPLICATION AND ARTIFACTS

To facilitate the reproducibility of this research, all hardware specifications, firmware source code, and attack scripts have been made publicly available. The experiments were conducted using an ESP32 do-it devkit v1 development board. All testing for this project was strictly confined to a fully isolated lab environment; no attacks were deployed against production networks, unauthorized devices, or third-party infrastructure. The project repository contains the C source code for each independent attack module and comprehensive documentation for flashing the microcontroller. These materials are accessible at: <https://github.com/cruzafonso3/SSR-Project>.

TABLE I
SUMMARY OF IMPLEMENTED ATTACKS, LIMITATIONS, AND MITIGATIONS

Attack Module	Success Status	Main Limitation	Network Mitigation
Deauthentication	Yes (WPA/WPA2 w/o PMF)	Requires channel synchronization and close physical proximity	Protected Management Frames (PMF)
Handshake Capture	Yes	CPU limits prohibit on-device cracking; requires offline execution	WPA3 (SAE) / Strong Passwords
DHCP Starvation	Yes (Pool exhausted)	Slow attack cycle due to forced driver reconnections	DHCP Snooping / 802.1X
Rogue AP (Evil Twin)	Yes (Credentials logged)	No internet passthrough; relies entirely on human interaction	User Awareness / Certificates
ARP Spoofing	Partial (Acts as DoS)	Severe buffer exhaustion and unstable packet forwarding	Dynamic ARP Inspection (DAI)

REFERENCES

- [1] M. Matuška, “Wi-fi attacks using ESP32,” in *Excel@FIT*. Brno University of Technology, Faculty of Information Technology, 2021. [Online]. Available: <https://excel.fit.vutbr.cz/submissions/2021/048/48.pdf>
- [2] M. Jalal, “MIM-attack-ESP32,” GitHub repository, 2025. [Online]. Available: <https://github.com/MuhamadJalalDev/MIM-attack-ESP32>
- [3] N. Sabbir, “ESP32-Evil-Twin-Captive-Portal,” GitHub repository, 2026. [Online]. Available: <https://github.com/neustarsabbir-max/ESP32-Evil-Twin-Captive-Portal>
- [4] Tanjib, “ESP32-ARP-Spoof.mpy-by-Tanjib,” GitHub repository, 2024. [Online]. Available: <https://github.com/MrEchoFi/ESP32-ARP-Spoof.mpy-by-Tanjib>
- [5] R. Stehlik, “Esp32 wi-fi penetration tool,” <https://risinek.github.io/esp32-wifi-penetration-tool/main/>, accessed: 2026-05-26.
- [6] CapibaraZero. (2023) DHCPGlutton. GitHub repository. [Online]. Available: <https://github.com/CapibaraZero/DHCPGlutton>
- [7] Electro-Gamma. (2024) esp32-handshake-capture. GitHub repository. [Online]. Available: <https://github.com/Electro-Gamma/esp32-handshake-capture>
- [8] Wikipedia contributors. (2024) IEEE 802.11w-2009 - Protected frames — Wikipedia the free encyclopedia. [Online; accessed 26-May-2026]. [Online]. Available: https://en.wikipedia.org/wiki/IEEE_802.11w-2009#Protected_frames